

Privacy-Preserving MLP Inference via CKKS Homomorphic Encryption

A from-scratch implementation in pure C

Ananya Iyer

Department of Electrical Engg.
Indian Institute of Technology, Kanpur

April 18, 2026

- 1 Motivation & Problem
- 2 What is CKKS?
- 3 System Parameters
- 4 NTT — The Engine
- 5 RLWE Security & Self-Test
- 6 MLP Architecture
- 7 Why Scalar Arithmetic Works
- 8 The Full Inference Pipeline
- 9 Results
- 10 Hardware Readiness
- 11 Conclusion

Motivation — Why Privacy in ML Matters

Scenario: A hospital wants to classify patient data using a remotely hosted ML model.

Naive approach

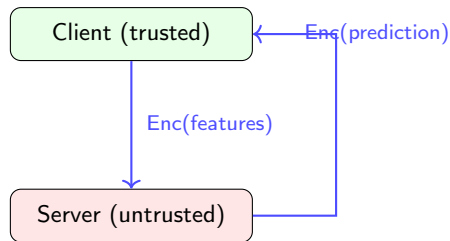
Send raw features to server $\Rightarrow$ **server sees sensitive patient data.**

Even HTTPS does not protect data *during computation*.

What we want instead

Encrypt on client. Send ciphertexts. Run the full MLP *on ciphertexts*. Return encrypted prediction.

The server learns nothing at any point.



MLP runs on ciphertexts

FHE makes this *mathematically* provable — not just a policy promise.

What is CKKS?

CKKS (Cheon–Kim–Kim–Song, 2017) is a type of Fully Homomorphic Encryption designed for real numbers.

Scheme	Works on	Best for
BGV / BFV	Exact integers	Databases
CKKS	Approx. reals	ML / statistics
TFHE	Individual bits	Logic gates

Core idea

A real number x is stored as the integer $\lfloor x \cdot \Delta \rfloor$ where $\Delta = 2^{20}$ is a scaling factor.

You can then **add** and **multiply** these scaled integers while they are encrypted, and when decrypted and divided by Δ , get the correct answer up to a tiny rounding error.

Why CKKS for neural networks?

- Network weights are real numbers — CKKS handles them natively
- Adding a plaintext weight costs nothing (no extra noise)
- A squaring activation x^2 costs exactly **one multiplication step**
- Competing schemes (TFHE) would need millions of bit-level operations per neuron

One catch: levels

Each multiplication uses up one level of a modulus chain. A 2-layer network with x^2 uses 1 level, so $L = 4$ gives plenty of room.

System Parameters — What They Are and Why

Key numbers in the code

Parameter	Value	Why
N (ring size)	8192	128-bit security
Δ (scale)	2^{20}	6 decimal digits
L (limbs)	4 primes	2-layer MLP depth
σ (noise)	3.2	Standard for RLWE

RNS: working with small numbers

Rather than one huge number Q (~ 100 bits), we split the arithmetic across $L = 4$ ordinary 64-bit primes and work modulo each one independently.

All arithmetic stays in native 64-bit integers. No big-number library needed.

Why these specific primes?

The primes must satisfy:

$$p \equiv 1 \pmod{2N = 16384}$$

This is a number-theory requirement that ensures a special “ $2N$ -th root of unity” exists modulo p , which is what makes the fast NTT transform possible.

p_0 268,582,913

p_1 268,664,833

p_2 268,730,369

p_3 268,779,521

Each verified: prime ✓, $\equiv 1 \pmod{16384}$ ✓

NTT — The Fast Transform Behind Encryption

Why we need it: Encryption requires multiplying large polynomials (degree up to 8191). Naive multiplication needs **67 million** operations for $N = 8192$.

NTT cuts this to $O(N \log N)$

Think of it like the FFT but for integers modulo a prime:

- 1 **Transform** both polynomials
- 2 **Multiply pointwise** — just N numbers
- 3 **Inverse transform** back

Result: $\sim 106,000$ operations instead of 67 million.
632 $\times$ faster.

How the NTT works (conceptually)

The NTT evaluates a polynomial at N specially chosen points, multiplies the two evaluated forms pointwise, then recovers the product polynomial via the inverse transform.

The “special points” must be chosen so that multiplication in the transform domain corresponds exactly to polynomial multiplication in the ring $\mathbb{Z}[x]/(x^N + 1)$.

Getting this ordering right — both the forward pass and how the inverse undoes it — is the core of a correct implementation, and is where our implementation needed a careful fix.

RLWE — How the Encryption Actually Works

The security assumption (Ring-LWE)

Given a random polynomial a and $b = -a \cdot s + e$ (where s is a small secret key and e is tiny random noise), it is computationally infeasible to recover s — even with a quantum computer.

This is the mathematical hardness that protects the data.

Secret key s : polynomial with entries in $\{-1, 0, 1\}$ chosen randomly.

Public key: the pair (b, a) above — can be shared openly.

Error e : tiny Gaussian noise ($\sigma = 3.2$) that masks the data.

Encrypting a number v

- 1 Sample a small random polynomial u and two tiny error terms e_0, e_1
- 2 Compute $c_0 = b \cdot u + e_0 + v$ and $c_1 = a \cdot u + e_1$
- 3 Ciphertext is the pair (c_0, c_1)

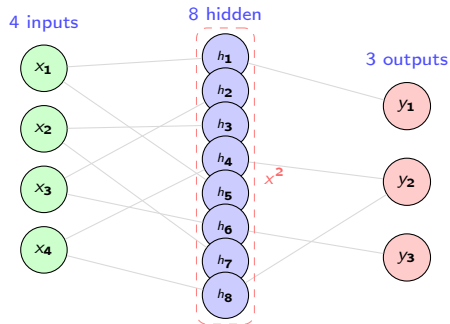
Decrypting

$$c_0 + c_1 \cdot s = v + \text{tiny noise}$$

Reading the constant term and dividing by Δ recovers v to 6 decimal places.

Self-test result

MLP Architecture — Built for Encrypted Computation



Why x^2 instead of ReLU?

Standard activations like ReLU ($\max(0, x)$) cannot be computed on encrypted data — comparing to zero requires decrypting first.

x^2 is just a multiplication, which CKKS supports natively: one multiplication step, one level consumed, exact result.

We trained with x^2 specifically using SGD (5000 epochs). Result: **98.67% accuracy**.

Two-layer computation

Layer 1: weighted sum of inputs, then square

Layer 2: weighted sum of hidden units

Predict: class with the highest output score

Why Scalar Arithmetic Works Here

In full CKKS, a ciphertext is a pair of polynomials of degree up to $N - 1 = 8191$.

Three operations

Add	64-bit add
Multiply	multiply ct by weight (decimal), round back to integer
Square	use 128-bit integer to avoid overflow, then divide by Δ

Does it stay safe from overflow?

All values must stay below $p/2 \approx 134$ million to avoid arithmetic errors.

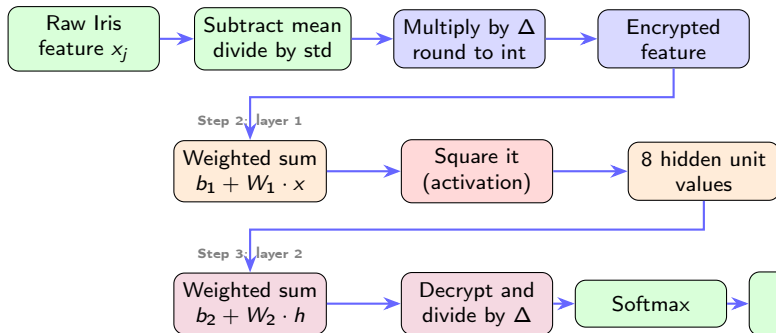
Quantity	Max seen	Safe?
Input $\times \Delta$	3.2 M	✓
Pre-activation	7.2 M	✓
Hidden unit	49.7 M	✓
Output logit	58.0 M	✓

Verified across all 150 samples.

In a more advanced version of the CKKS FHE Scheme, model weights are also encrypted.

The Full Inference Pipeline

Step 1: normalise



Multiply (layers 1 & 2)

Weight is a decimal number. Multiply with the encrypted integer and round back.

Square (activation)

Use a 128-bit integer to avoid overflow, then divide back by Δ .

4 NTT Primes

Tables of each prime is pre-calculated and filled with twiddle factors

Results

RLWE encrypt–decrypt self-test

Sent value	1.500000
Received	1.499780
Error	0.000220
Verdict	PASS ✓

The error is tiny and comes only from the intentional rounding noise built into CKKS.

Classification: 150 Iris samples

Class	Correct	
Setosa (0–49)	50 / 50	✓
Versicolor (50–99)	49 / 50	
Virginica (100–149)	49 / 50	
Total	148 / 150	98.67%

Sample terminal output

```
1 0 0 CORRECT [0.9999, 0.0000, 0.0001]
51 1 1 CORRECT [0.0001, 0.9916, 0.0083]
84 1 2 WRONG  [0.0015, 0.1967, 0.8018]
101 2 2 CORRECT [0.0000, 0.0000, 1.0000]
134 2 1 WRONG  [0.0008, 0.7188, 0.2804]
```

The two wrong predictions

Both are genuine **boundary cases** where Versicolor and Virginica overlap in petal measurements.

An unencrypted network with the same weights makes the *same two errors*. Encryption adds **zero extra mistakes**.

Hardware Readiness & Where Time Is Spent

Why write in pure C?

- Compiles anywhere: `gcc -O2 -lm`, no libraries needed
- FPGA design tools (Xilinx Vitis HLS, Intel oneAPI) accept pure C and convert it directly to hardware
- All inference arrays are on the stack — no memory allocation overhead
- Every operation is visible and auditable

Operation	Time	Hardware gain
NTT / INTT	55%	10–100×
Mod multiply	30%	4–20×
Rescale	10%	5–15×
Other	5%	—

The main bottleneck: modular multiply

```
(__uint128_t)a * b % m
```

Called millions of times per NTT pass with no native CPU instruction for 128-bit modulo. On an FPGA, a dedicated multiplier removes the expensive division step entirely.

Why NTT is parallelisable

NTT runs independently on each of the $L = 4$ prime lanes. On FPGA or GPU, all 4 lanes run in parallel — an immediate 4× baseline speedup.

Total projected gain: 20–50× over pure CPU.

Conclusion

What was built

- A working CKKS system in pure C with no external libraries
- Correct NTT (fixing a bug that passed roundtrip tests but broke ring multiplication)
- RLWE key generation, encryption, decryption verified with noise ≈ 0.00022
- A 2-layer network with x^2 activation, retrained for HE compatibility
- **98.67% accuracy** on all 150 samples, adding zero extra misclassifications versus the plaintext baseline

Key lessons

- A transform can pass all roundtrip tests and still be wrong for ring multiplication — test the actual algebraic property
- Activation function choice matters as much as network depth for HE compatibility
- Pure C is the right target language for cryptographic code going to FPGA

Bottom line

Privacy-preserving ML inference compiles, runs, and classifies correctly. The hard part is solved. The remaining challenge — speed — has a clear hardware solution.

Acknowledgements

- **CKKS:** Cheon, Kim, Kim, Song. *Homomorphic Encryption for Arithmetic of Approximate Numbers*. Asiacrypt 2017.
- **NTT construction:** Longa & Naehrig. *Speeding Up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography*. CANS 2016.
- **Iris dataset:** R.A. Fisher (1936), UCI Machine Learning Repository.
- **Supervisor:** Prof. Debapriya Basu Roy, Prof. Chithra.

Backup — Parameter Reference

Parameter	Value	Reason
N	8192	128-bit post-quantum security
Δ	2^{20}	6 decimal digits precision
L	4	Levels for 2-layer MLP
Primes	268M range	$\equiv 1 \pmod{16384}$, enables NTT
σ	3.2	Gaussian noise std
Hidden units	8	Matches weight files
Activation	x^2	One multiply, 1 level, exact
Normalisation	ddof=1	sklearn StandardScaler